

15 mai 2026

Table des matières

Préface

La **science des données** est une discipline qui combine mathématiques, statistiques et informatique pour extraire des connaissances à partir de données. Elle se distingue de la statistique classique par son échelle (volume de données), ses outils (programmation, apprentissage automatique) et son orientation pratique (décisions, produits, prédictions).

Ce cours est conçu pour des étudiants de Licence ayant des bases en Python et en statistiques. Chaque chapitre part d'un **jeu de données réel**, introduit les concepts et les met en œuvre en Python. L'objectif est de rendre l'étudiant autonome pour mener une analyse de données complète, de l'import à la communication des résultats.

Prérequis. Programmation Python de base (variables, boucles, fonctions), notions de statistique descriptive et de probabilités.

Jeux de données utilisés. Iris, Titanic, Tips (Seaborn), California Housing, 20 Newsgroups, AirPassengers — tous librement accessibles.

Références.

- VANDERPLAS — *Python Data Science Handbook*, O'Reilly (disponible gratuitement en ligne).
- MCKINNEY — *Python for Data Analysis*, O'Reilly.
- GÉRON — *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly.
- JAMES, Witten, Hastie, Tibshirani — *An Introduction to Statistical Learning (ISLR)*, Springer (gratuit).

Chapitre 1

Le Pipeline de la Science des Données

En 1854, le médecin londonien John Snow trace sur une carte de Soho l'emplacement de chaque décès par choléra. Les points s'agglutinent autour d'une pompe à eau de Broad Street. Snow demande aux autorités de retirer la manivelle de la pompe ; l'épidémie s'éteint en quelques jours. Personne, à l'époque, ne savait que la maladie se transmet par l'eau : la théorie microbienne n'existe pas encore. Snow ne démontre rien au sens médical du terme ; il fait quelque chose de plus moderne. Il pose une question (qui meurt ?), collecte des données (les registres de décès), les géolocalise (la carte), repère un motif (la concentration spatiale), formule une hypothèse causale (l'eau contaminée) et agit sur le monde (la manivelle). C'est, à cent soixante-dix ans près, le pipeline standard d'un projet de science des données. Ce chapitre déroule chacun de ces maillons — question, ingestion, exploration, modélisation, communication — et explique pourquoi sauter l'un d'eux suffit à transformer un projet ambitieux en tableau de bord que personne ne regarde.

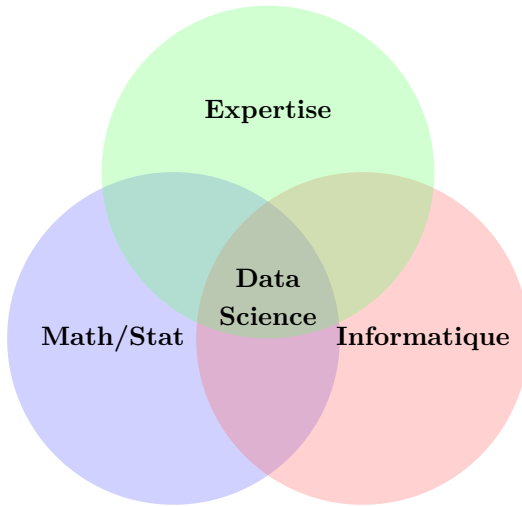
1.1 Qu'est-ce que la science des données ?

Définition 1.1 (Science des données). La **science des données** (data science) est une discipline interdisciplinaire qui utilise des méthodes scientifiques, des algorithmes et des systèmes pour extraire des connaissances et des informations à partir de données structurées et non structurées.

Intuition

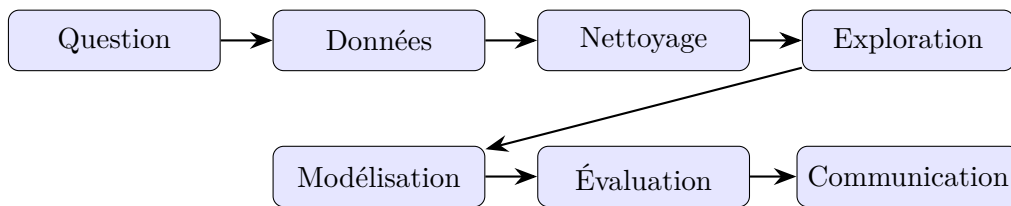
La science des données se situe à l'intersection de trois domaines :

- **Mathématiques et Statistiques** : modélisation, inférence, probabilités.
- **Informatique** : programmation, algorithmes, bases de données.
- **Expertise métier** : compréhension du domaine d'application.



1.2 Le pipeline typique

Un projet de science des données suit généralement un pipeline en plusieurs étapes :



1. **Formulation de la question** : définir clairement le problème.
2. **Collecte des données** : import depuis des fichiers, bases de données, API.
3. **Nettoyage** : traiter les valeurs manquantes, les doublons, les erreurs.
4. **Exploration** (EDA) : visualiser et résumer les données.
5. **Modélisation** : appliquer des modèles statistiques ou de machine learning.
6. **Évaluation** : mesurer la performance du modèle.
7. **Communication** : présenter les résultats de manière claire.

1.3 L'écosystème Python pour la data science

Bibliothèques essentielles

Bibliothèque	Rôle
<MINTED>	Calcul numérique, tableaux
<MINTED>	Manipulation de données tabulaires
<MINTED>	Visualisation de base
<MINTED>	Visualisation statistique
<MINTED>	Apprentissage automatique
<MINTED>	Statistiques et optimisation

Python — Installation et imports

<MINTED>

Sortie

```
NumPy: 1.26.4
Pandas: 2.2.1
Seaborn: 0.13.2
```

1.4 Premier exemple complet : le jeu de données Iris

Illustrons le pipeline complet sur le jeu de données **Iris** de Fisher (1936), qui contient 150 mesures de fleurs d'iris réparties en 3 espèces.

Python — Charger et explorer Iris

<MINTED>

Sortie

```
(150, 5)
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1                3.5                1.4                0.2        0
1                4.9                3.0                1.4                0.2        0
2                4.7                3.2                1.3                0.2        0
3                4.6                3.1                1.5                0.2        0
4                5.0                3.6                1.4                0.2        0
```

Python — Statistiques descriptives

<MINTED>

Sortie

```

  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  ta
count            150.00            150.00            150.00            150.00  15
mean              5.84              3.06              3.76              1.20
std              0.83              0.44              1.77              0.76
```

min	4.30	2.00	1.00	0.10
25%	5.10	2.80	1.60	0.30
50%	5.80	3.00	4.35	1.30
75%	6.40	3.30	5.10	1.80
max	7.90	4.40	6.90	2.50

Valeurs manquantes :

sepal length (cm) 0

sepal width (cm) 0

petal length (cm) 0

petal width (cm) 0

target	0
--------	---

Python — Visualisation

<MINTED>

Python — Modélisation simple

<MINTED>

Sortie

Accuracy : 100.00%

1.5 Types de problèmes en data science

Supervisé Régression, Classification

Non supervisé

Renforcement
Agent, Récompense

- **Apprentissage supervisé** : on dispose d'un label y pour chaque observation $\mathbf{x}$.
 - *Régression* : $y \in \mathbb{R}$ (prédire un prix, une température).
 - *Classification* : $y \in \{0, 1, \dots, K - 1\}$ (détecter un spam, un cancer).
- **Apprentissage non supervisé** : pas de label, on cherche des structures (clusters, composantes principales).
- **Apprentissage par renforcement** : un agent apprend par essai-erreur dans un environnement.

1.6 Bonnes pratiques

Bonne pratique

1. **Reproductibilité** : fixer les graines aléatoires (<MINTED>), versionner le code.
2. **Séparation train/test** : ne jamais évaluer sur les données d'entraînement.
3. **Documentation** : commenter le code, noter les hypothèses.
4. **Itération** : commencer simple, complexifier progressivement.

Attention

Ne jamais regarder les données de test avant d'avoir finalisé son modèle. Le *data leakage* (fuite de données) est l'erreur la plus courante et la plus dangereuse en data science.

1.7 Exercices

Exercice 1.1 (★). Charger le jeu de données <MINTED> de scikit-learn. Afficher ses dimensions, les noms des variables et les statistiques descriptives.

Exercice 1.2 (★). Sur le jeu Iris, créer un nuage de points (*scatter plot*) de la largeur du sépale en fonction de la longueur du sépale, coloré par espèce.

Exercice 1.3 (★★). Appliquer un classifieur k -plus proches voisins au jeu Wine avec $k = 3, 5, 7$. Comparer les précisions sur un jeu de test de 30%.

Exercice 1.4 (★★). Écrire une fonction <MINTED> qui charge un jeu de données scikit-learn, le divise en train/test, entraîne un KNN avec k voisins et retourne l'accuracy.

Résumé du chapitre

- La science des données suit un pipeline : Question → Données → Nettoyage → Exploration → Modélisation → Évaluation → Communication.
- L'écosystème Python offre des outils puissants : <MINTED>, <MINTED>, <MINTED>.
- Trois grandes familles : supervisé, non supervisé, renforcement.
- Règles d'or : reproductibilité, séparation train/test, commencer simple.

Chapitre 2

Manipulation de Données avec Pandas

2.1 Introduction à Pandas

Pandas est la bibliothèque centrale de la data science en Python. Elle fournit deux structures de données fondamentales : le **<MINTED>** (vecteur indexé) et le **<MINTED>** (tableau indexé).

Définition 2.1 (DataFrame). Un **DataFrame** est un tableau bidimensionnel avec des étiquettes pour les lignes (index) et les colonnes. Chaque colonne est une **<MINTED>** de même longueur.

2.2 Création de DataFrames

Python — Création

<MINTED>

Sortie

	Nom	Âge	Salaire	Ville
0	Alice	25	45000	Paris
1	Bob	30	55000	Lyon
2	Charlie	35	70000	Paris
3	Diana	28	52000	Marseille

Shape : (4, 4)

Types :

Nom object

Âge int64

Salaire int64

Ville object

dtype: object

2.3 Chargement de données

Python — Lecture de fichiers

<MINTED>

Sortie

Dimensions : (244, 7)

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3

2.4 Sélection et indexation

Méthodes de sélection

Syntaxe	Description
<MINTED>	Sélectionner une colonne (Series)
<MINTED>	Sélectionner plusieurs colonnes
<MINTED>	Sélection par étiquette
<MINTED>	Sélection par position
<MINTED>	Filtrage booléen

Python — Sélection

<MINTED>

Sortie

--- Colonne unique ---

0 16.99

1 10.34

2 21.01

Name: total_bill, dtype: float64

--- Pourboires > 5\$: 17 lignes ---

	total_bill	tip	sex	smoker	day	time	size
23	39.42	7.58	Male	No	Sat	Dinner	4
44	30.40	5.60	Male	No	Sun	Dinner	4
47	32.40	6.00	Male	No	Sun	Dinner	4

--- loc : femmes, dimanche ---

	total_bill	tip
0	16.99	1.01
5	25.29	4.71
15	21.58	3.92

2.5 Modification et ajout de colonnes

Python — Nouvelles colonnes

<MINTED>

Sortie

```

    total_bill  tip  tip_pct
0         16.99  1.01     5.9
1         10.34  1.66    16.1
2         21.01  3.50    16.7
3         23.68  3.31    14.0
4         24.59  3.61    14.7

```

```

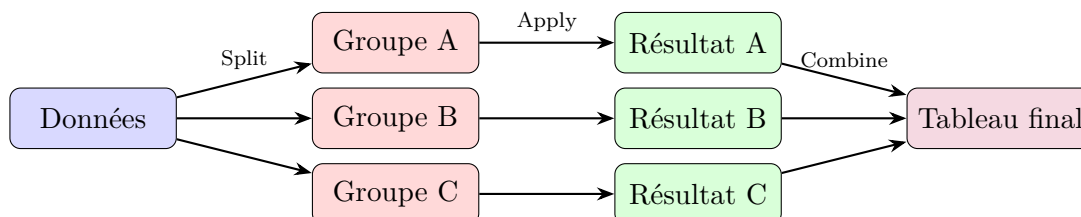
bill_category
Moyen      111
Bas         69
Élevé       64
Name: count, dtype: int64

```

2.6 Agrégation avec groupby

Définition 2.2 (Split-Apply-Combine). Le paradigme **split-apply-combine** consiste à :

1. **Split** : diviser les données en groupes.
2. **Apply** : appliquer une fonction (somme, moyenne, comptage...) à chaque groupe.
3. **Combine** : combiner les résultats.



Python — groupby

<MINTED>

Sortie

```

--- Moyenne par jour ---
    total_bill  tip
day
Thur         17.68  2.77
Fri         17.15  2.73
Sat         20.44  2.99
Sun         21.41  3.26

```

```

--- Agrégations multiples ---
      nb_repas  addition_moy  pourboire_moy
day  time
Thur Lunch      61         17.66         2.77
     Dinner      1         18.78         3.00
Fri  Lunch       7         12.85         2.38
     Dinner     12         19.66         2.94
Sat  Dinner     87         20.44         2.99
Sun  Dinner     76         21.41         3.26

```

2.7 Tableau croisé dynamique

Python — pivot_table

<MINTED>

Sortie

sex	Female	Male
day		
Thur	2.58	2.88
Fri	2.78	2.69
Sat	2.80	3.08
Sun	3.37	3.22

2.8 Jointures

Python — merge

<MINTED>

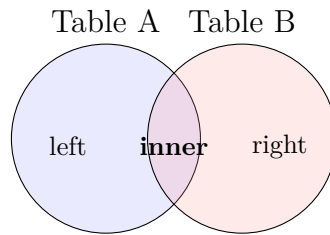
Sortie

```

--- Inner join ---
  client_id  nom  commande_id  montant
0         1  Alice         101      150
1         2   Bob         102      200
2         2   Bob         103       80

--- Left join ---
  client_id  nom  commande_id  montant
0         1  Alice         101.0    150.0
1         2   Bob         102.0    200.0
2         2   Bob         103.0     80.0
3         3 Charlie          NaN     NaN
4         4  Diana          NaN     NaN

```

$$\text{outer} = A \cup B$$

2.9 Tri et classement

Python — Tri

<MINTED>

Sortie

```

total_bill  tip  day
170      50.81 10.00 Sat
212      48.33  9.00 Sat
23       39.42  7.58 Sat
59       48.27  6.73 Sat
183      23.17  6.50 Sun

total_bill  tip  tip_rank
0       16.99  1.01      224
1       10.34  1.66      188
2       21.01  3.50       52
3       23.68  3.31       63
4       24.59  3.61       46
    
```

2.10 Opérations sur les chaînes

Python — Accesseur .str

<MINTED>

Sortie

```

0    Alice Dupont
1    Bob Martin
2    Charlie Petit
dtype: object
0    False
1     True
2    False
dtype: bool
    
```

2.11 Exercices

Exercice 2.1 (*). Charger le jeu **<MINTED>** de Seaborn. Afficher le nombre de repas par jour de la semaine et par moment (déjeuner/dîner).

Exercice 2.2 (*). Créer un DataFrame avec 5 étudiants, leurs notes en maths et en physique. Ajouter une colonne « moyenne » et trier par moyenne décroissante.

Exercice 2.3 (**). Sur le jeu Tips, calculer pour chaque combinaison (sexe, fumeur) : le nombre de repas, l'addition moyenne et le pourcentage moyen de pourboire.

Exercice 2.4 (**). Créer deux DataFrames : un avec des produits (id, nom, catégorie), un avec des ventes (id_produit, quantité, date). Les joindre et calculer le chiffre d'affaires par catégorie.

Exercice 2.5 (***). Écrire une fonction qui prend un DataFrame et retourne un rapport automatique : nombre de lignes, types de colonnes, pourcentage de valeurs manquantes par colonne, nombre de doublons.

Fonctions Pandas essentielles

- **<MINTED>**, **<MINTED>** — import/export
- **<MINTED>**, **<MINTED>**, **<MINTED>** — exploration
- **<MINTED>**, **<MINTED>** — sélection
- **<MINTED>** — agrégation
- **<MINTED>** — jointures
- **<MINTED>**, **<MINTED>** — tri

Chapitre 3

Visualisation des Données

Intuition

La visualisation est le premier outil du data scientist. Un bon graphique révèle des structures, des anomalies et des relations qu'aucun tableau de chiffres ne peut montrer aussi clairement. Comme le disait John Tukey : « The greatest value of a picture is when it forces us to notice what we never expected to see. »

3.1 La grammaire des graphiques

La **grammaire des graphiques**, proposée par Leland Wilkinson, décompose tout graphique en couches :

1. **Données** : le jeu de données source.
2. **Esthétiques** (*aesthetics*) : les variables mappées sur les axes, couleurs, tailles.
3. **Géométries** : le type de représentation (points, barres, lignes).
4. **Facettes** : le découpage en sous-graphiques.
5. **Statistiques** : les transformations appliquées (comptage, moyenne).
6. **Coordonnées** : le système de coordonnées (cartésien, polaire).
7. **Thème** : l'apparence générale (polices, grille, fond).

Seaborn et Matplotlib implémentent ces concepts en Python.

3.2 Matplotlib : les fondamentaux

3.2.1 Graphique en ligne

Python

<MINTED>

Sortie

Un graphique avec deux courbes (sinus en bleu, cosinus en tirets corail) sur $[0, 2\pi]$, avec légende, grille et titres.

3.2.2 Nuage de points et histogramme**Python**

```
<MINTED>
```

Sortie

Deux sous-graphiques : à gauche un nuage de points montrant une relation positive entre addition et pourboire ; à droite un histogramme légèrement asymétrique (queue droite) des additions.

3.2.3 Diagramme en barres**Python**

```
<MINTED>
```

Sortie

Diagramme en barres colorées avec les valeurs annotées : Fri 2.73, Thur 2.77, Sat 2.99, Sun 3.26.

3.3 Seaborn : visualisation statistique**3.3.1 Distributions : histplot et violinplot****Python**

```
<MINTED>
```

Sortie

À gauche : histogrammes superposés avec courbes KDE pour homme et femme. À droite : violin plots séparés par sexe pour chaque jour, montrant que les hommes ont des additions légèrement plus élevées.

3.3.2 Boîtes à moustaches et heatmap**Python**

```
<MINTED>
```

Sortie

Boxplots montrant des distributions similaires entre fumeurs et non-fumeurs. Heat-map : corrélation $\text{total_bill}/\text{tip} = 0.68$, $\text{total_bill}/\text{size} = 0.60$, $\text{tip}/\text{size} = 0.49$.

3.3.3 Pairplot et relplot

Python

<MINTED>

Sortie

Matrice 4×4 de graphiques : diagonale = densités KDE par espèce, hors-diagonale = nuages de points. On distingue clairement *setosa* des deux autres espèces.

Python

<MINTED>

Sortie

Deux panneaux (Lunch / Dinner) avec nuages de points colorés par statut fumeur et différenciés par forme selon le sexe.

3.3.4 catplot

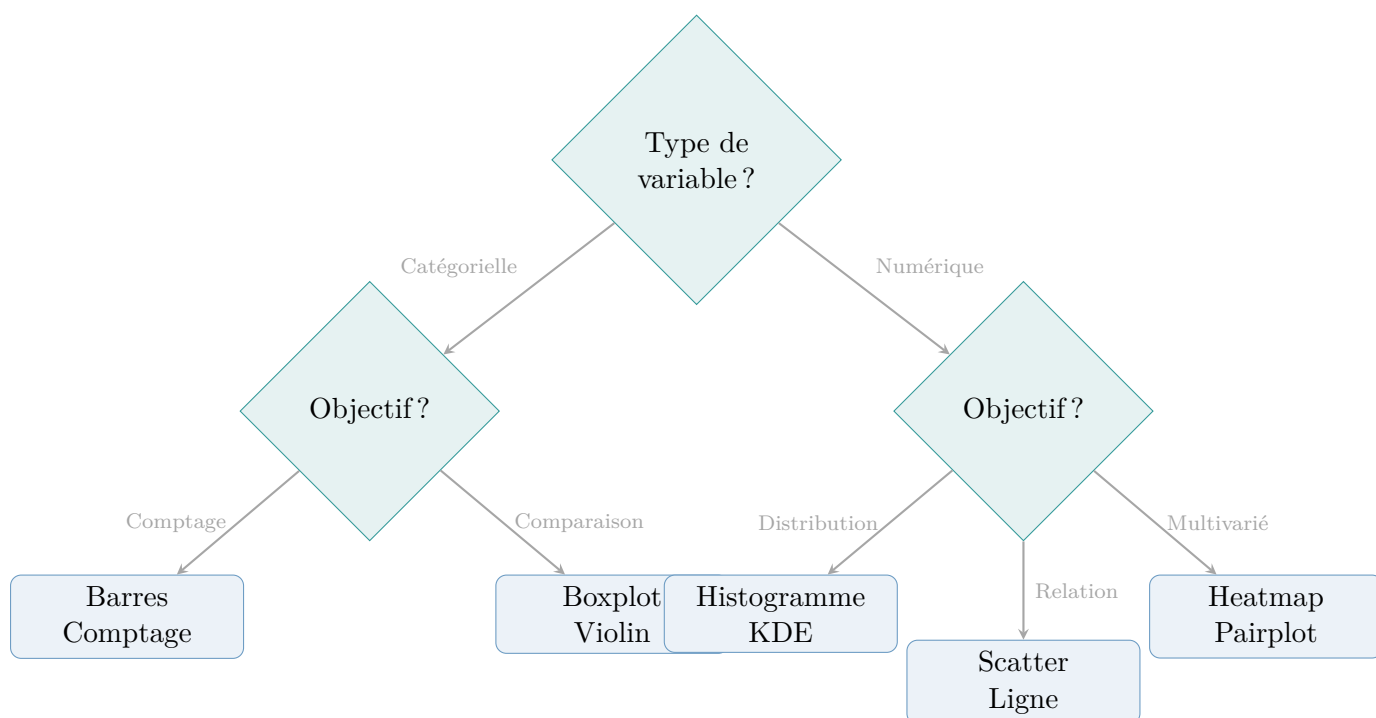
Python

<MINTED>

Sortie

Quatre groupes de boxplots organisés en deux colonnes (Lunch, Dinner), comparés par sexe et par jour.

3.4 Choisir le bon type de graphique



3.5 Palettes de couleurs et sauvegarde

Python

<MINTED>

Bonne pratique

Utilisez toujours la palette <MINTED> pour garantir l'accessibilité de vos graphiques. Sauvegardez en PDF pour les rapports et en PNG (300 dpi) pour les présentations.

3.6 Erreurs courantes en visualisation

Attention

- **Axes tronqués** : ne pas commencer l'axe y à zéro dans un diagramme en barres exagère les différences.
- **Échelles trompeuses** : utiliser deux axes y avec des échelles différentes peut suggérer de fausses corrélations.
- **Graphiques 3D** : les perspectives déforment la perception des valeurs. Préférez le 2D.
- **Trop de catégories** : un camembert avec 15 parts est illisible. Utilisez un diagramme en barres.
- **Absence de légende et de titres** : tout graphique doit être compréhensible seul.

3.7 Exercices

Exercice 3.1 (*). Chargez le jeu `<MINTED>` et créez un histogramme de la variable `<MINTED>` avec 20 intervalles. Ajoutez un titre, des labels d'axes et une courbe KDE superposée avec Seaborn.

Exercice 3.2 (*). Créez un diagramme en barres horizontales montrant le nombre de repas par jour dans le jeu `<MINTED>`. Triez les barres par ordre décroissant.

Exercice 3.3 (**). À l'aide du jeu `<MINTED>`, créez une figure avec 4 sous-graphiques (2×2) montrant la distribution de chaque variable numérique, colorée par espèce. Utilisez `<MINTED>` avec `<MINTED>`.

Exercice 3.4 (**). Créez un `<MINTED>` de type `<MINTED>` montrant la distribution de `<MINTED>` par `<MINTED>`, facetté par `<MINTED>`, avec la couleur déterminée par `<MINTED>`. Interprétez les différences observées.

Exercice 3.5 (* * *). Reproduisez le graphique suivant : une figure à deux panneaux. Le panneau gauche montre un scatter plot de `<MINTED>` vs `<MINTED>` avec une droite de régression (`<MINTED>`). Le panneau droit montre les résidus de cette régression (calculez-les avec NumPy : `<MINTED>` et `<MINTED>`). Commentez la qualité de l'ajustement.

Exercice 3.6 (* * *). Prenez le jeu `<MINTED>` et créez une heatmap de la matrice de corrélation *par espèce* (trois heatmaps dans une figure 1×3). Comparez les structures de corrélation entre espèces.

Fonctions clés

Résumé du chapitre – Visualisation des données

- Matplotlib : `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`.
- Seaborn (distributions) : `<MINTED>`, `<MINTED>`, `<MINTED>`.
- Seaborn (relations) : `<MINTED>`, `<MINTED>`, `<MINTED>`.
- Seaborn (catégoriel) : `<MINTED>`, `<MINTED>`, `<MINTED>`.
- Seaborn (multivarié) : `<MINTED>`.
- Personnalisation : `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`.
- Sauvegarde : `<MINTED>`.
- Règle d'or : choisir le graphique adapté au type de variable et à l'objectif d'analyse.

Chapitre 4

Analyse Exploratoire des Données (EDA)

Intuition

L'analyse exploratoire des données (EDA) est une philosophie d'analyse initiée par John Tukey dans les années 1970. Plutôt que de tester des hypothèses prédéfinies, l'EDA invite à « laisser parler les données » en les explorant systématiquement par des résumés numériques et des visualisations.

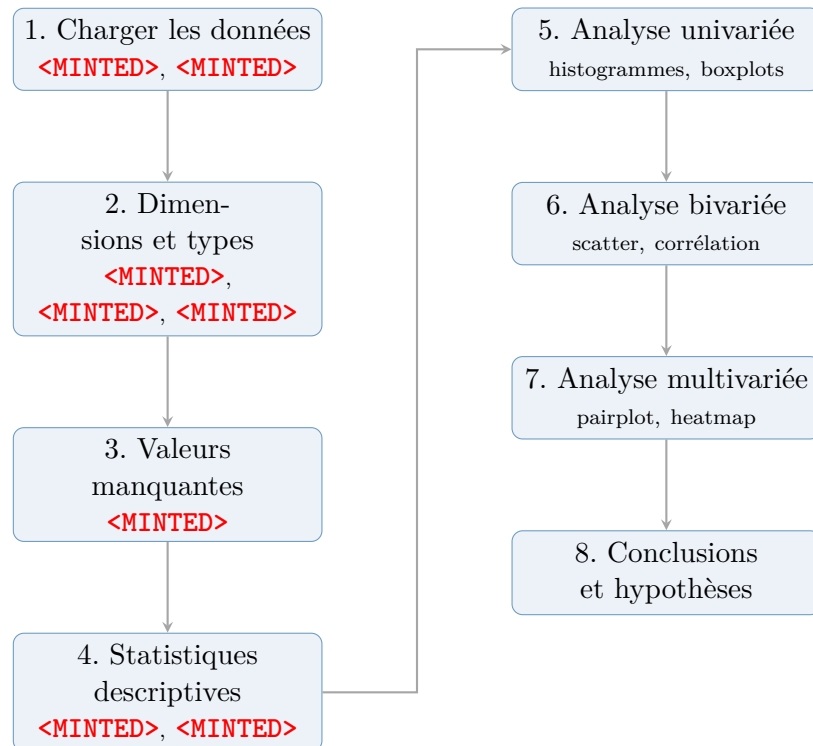
4.1 Qu'est-ce que l'EDA ?

Définition 4.1 (Analyse exploratoire des données). L'EDA est une approche d'analyse qui utilise des techniques visuelles et quantitatives pour comprendre la structure, les anomalies et les relations dans un jeu de données *avant* toute modélisation formelle.

Les objectifs principaux de l'EDA sont :

- Découvrir la **structure** des données (dimensions, types, valeurs manquantes).
- Identifier les **distributions** et les **tendances centrales**.
- Repérer les **anomalies** et les **valeurs aberrantes**.
- Explorer les **relations** entre variables.
- Formuler des **hypothèses** à tester ultérieurement.

4.2 Processus systématique d'EDA



4.3 Étude de cas : le jeu Titanic

4.3.1 Chargement et première inspection

```
Python
<MINTED>
```

```
Sortie
<MINTED>
```

4.3.2 Valeurs manquantes

```
Python
<MINTED>
```

```
Sortie
<MINTED>
```

Remarque 4.2. La variable <MINTED> a 77 % de valeurs manquantes : elle sera probablement inutilisable sans imputation lourde. La variable <MINTED> a environ 20 % de manquants, ce qui est gérable.

4.3.3 Statistiques descriptives

Python

<MINTED>

Sortie

<MINTED>

4.4 Analyse univariée

Python

<MINTED>

Sortie

Quatre graphiques : l'âge suit approximativement une distribution normale centrée vers 29 ans ; la troisième classe est la plus représentée (491) ; 62 % des passagers n'ont pas survécu ; le tarif est très asymétrique avec une longue queue droite.

4.5 Analyse bivariée

4.5.1 Survie par sexe et par classe

Python

<MINTED>

Sortie

Les femmes ont un taux de survie de 74 % contre 19 % pour les hommes. La 1ère classe a 63 % de survie, la 2ème 47 % et la 3ème 24 %. La combinaison montre que les femmes de 1ère classe ont survécu à près de 97 %.

4.5.2 Corrélations et table de contingence

Python

<MINTED>

Sortie

<MINTED>

4.6 Analyse multivariée

Python

<MINTED>

Sortie

Heatmap montrant les corrélations : la survie est négativement corrélée avec la classe (-0.34) et positivement avec le tarif (0.26). La classe et le tarif sont fortement anti-corrélés (-0.55).

4.7 Détection des valeurs aberrantes

Définition 4.3 (Méthode IQR). Une observation est considérée comme aberrante si elle se trouve en dehors de l'intervalle :

$$[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}]$$

où $\text{IQR} = Q_3 - Q_1$ est l'écart interquartile.

Python

<MINTED>

Sortie

<MINTED>

Remarque 4.4. 116 passagers (13 %) ont un tarif considéré comme aberrant par la méthode IQR. Ce sont principalement des passagers de première classe. Il ne faut pas les supprimer automatiquement : ce sont des valeurs légitimes qui reflètent la structure des données.

4.8 Exercices

Exercice 4.1 (★). Chargez le jeu <MINTED> et affichez les 5 premières lignes, les dimensions, les types de colonnes et le nombre de valeurs manquantes par colonne.

Exercice 4.2 (★). Calculez le taux de survie global, puis le taux de survie par port d'embarquement (<MINTED>). Quelle est la lettre du port avec le meilleur taux de survie ?

Exercice 4.3 (★★). Créez un graphique avec <MINTED> de type <MINTED> montrant le taux de survie par classe et par sexe. Ajoutez un titre informatif. Interprétez les résultats en termes de priorité d'évacuation.

Exercice 4.4 (★★). Créez un histogramme de l'âge des passagers, séparé par survie (<MINTED>). Les survivants sont-ils en moyenne plus jeunes ? Vérifiez avec <MINTED>.

Exercice 4.5 (★★★). Appliquez la méthode IQR pour détecter les outliers dans la variable <MINTED>. Combien sont-ils ? Créez un boxplot annoté montrant les bornes IQR et les outliers. Discutez de la pertinence de supprimer ces observations.

Exercice 4.6 (★ ★ ★). Réalisez une EDA complète du jeu <MINTED> (<MINTED>). Votre analyse doit inclure : (a) inspection générale, (b) traitement des valeurs manquantes, (c) distributions univariées, (d) analyse bivariée par espèce, (e) heatmap de corrélation. Concluez en formulant 3 hypothèses testables.

Fonctions clés

Résumé du chapitre – Analyse exploratoire des données

- Inspection : <MINTED>, <MINTED>, <MINTED>, <MINTED>.
- Manquants : <MINTED>, <MINTED>.
- Descriptif : <MINTED>, <MINTED>.
- Univarié : <MINTED>, <MINTED>, <MINTED>.
- Bivarié : <MINTED>, <MINTED>, <MINTED>.
- Multivarié : <MINTED>, <MINTED>, <MINTED>.
- Outliers (IQR) : aberrant si $x < Q_1 - 1.5 \cdot \text{IQR}$ ou $x > Q_3 + 1.5 \cdot \text{IQR}$.
- Philosophie Tukey : explorer d'abord, modéliser ensuite.

Chapitre 5

Nettoyage et Préparation des Données

Intuition

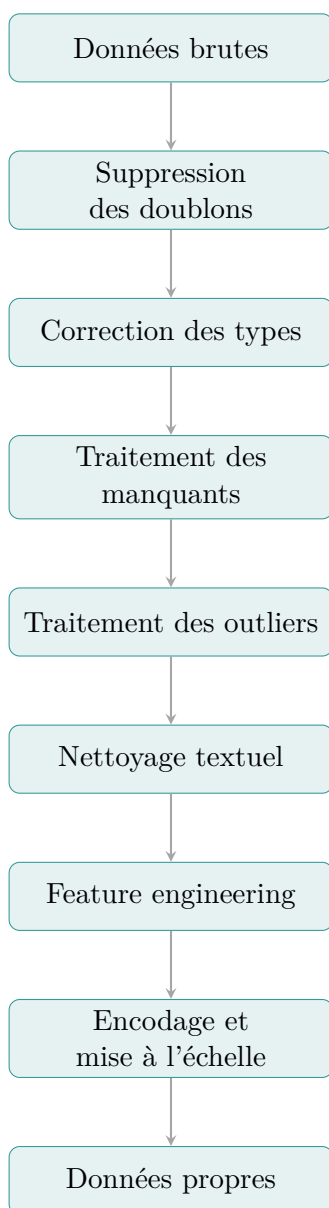
On estime que les data scientists consacrent environ 80 % de leur temps au nettoyage et à la préparation des données. Des données mal préparées mènent inévitablement à des modèles erronés : « Garbage in, garbage out. »

5.1 Pourquoi nettoyer les données ?

Les données réelles sont rarement propres. On rencontre fréquemment :

- des **valeurs manquantes** (capteurs défaillants, champs non remplis) ;
- des **doublons** (saisies multiples, fusions de tables) ;
- des **types incorrects** (nombres stockés comme texte) ;
- des **valeurs aberrantes** (erreurs de saisie) ;
- des **incohérences textuelles** (majuscules, espaces, abréviations).

5.2 Pipeline de nettoyage



5.3 Étude de cas : le jeu Titanic

Python

<MINTED>

Sortie

<MINTED>

5.4 Valeurs manquantes

5.4.1 Types de données manquantes

Définition 5.1 (Classification des valeurs manquantes). • **MCAR** (*Missing Completely At Random*) : le caractère manquant ne dépend d’aucune variable.

- **MAR** (*Missing At Random*) : le caractère manquant dépend d’autres variables observées.
- **MNAR** (*Missing Not At Random*) : le caractère manquant dépend de la valeur elle-même.

Exemple 5.2. Dans le Titanic, les cabines (<MINTED>) manquent surtout pour les passagers de 3ème classe (MAR). L’âge pourrait être MCAR ou MAR selon que les passagers sans billet complet n’avaient pas d’âge enregistré.

5.4.2 Détection et traitement

Python

<MINTED>

Sortie

<MINTED>

Bonne pratique

L’imputation par la médiane *conditionnée* (ici par classe) est préférable à la médiane globale car elle préserve la structure des sous-groupes. Pour des données temporelles, utilisez <MINTED> ou <MINTED>.

5.5 Doublons

Python

<MINTED>

Sortie

<MINTED>

5.6 Conversion de types

Python

<MINTED>

Sortie

<MINTED>

5.7 Traitement des valeurs aberrantes

Python

<MINTED>

Sortie

<MINTED>

Attention

Ne supprimez jamais les outliers sans réflexion. Un tarif de 512\$ peut être légitime (suite de luxe). L'écrtage est souvent préférable à la suppression. Documentez toujours vos choix.

5.8 Nettoyage textuel

Python

<MINTED>

Sortie

<MINTED>

5.9 Feature engineering

Python

<MINTED>

Sortie

<MINTED>

5.10 Encodage des variables catégorielles

Python

<MINTED>

Sortie

<MINTED>

Attention

Le **label encoding** impose un ordre artificiel aux catégories. Il ne convient que pour les variables ordinales ou les algorithmes à base d'arbres. Pour les modèles linéaires, préférez le **one-hot encoding**.

5.11 Mise à l'échelle

Définition 5.3 (Standardisation et normalisation). • **Standardisation** (StandardScaler) : $z = \frac{x - \mu}{\sigma}$, résultat de moyenne 0 et d'écart-type 1.

- **Normalisation** (MinMaxScaler) : $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$, résultat dans $[0, 1]$.

Python

<MINTED>

Sortie

<MINTED>

Bonne pratique

Appliquez la mise à l'échelle *après* la séparation train/test pour éviter la fuite d'information (*data leakage*). Utilisez <MINTED> sur le train et <MINTED> sur le test.

5.12 Exercices

Exercice 5.1 (*). Chargez le jeu <MINTED>. Affichez le nombre et le pourcentage de valeurs manquantes par colonne. Quelles colonnes ont plus de 50 % de manquants ?

Exercice 5.2 (*). Supprimez les doublons du jeu <MINTED> (s'il y en a). Puis convertissez la colonne <MINTED> en variable catégorielle avec <MINTED>.

Exercice 5.3 (**). Imputez les valeurs manquantes de <MINTED> par la médiane conditionnée par <MINTED> et <MINTED> (6 groupes). Comparez la distribution de l'âge avant et après imputation avec des histogrammes superposés.

Exercice 5.4 (**). Créez les variables suivantes : <MINTED> (extrait du nom avec une expression régulière), <MINTED> (vrai si <MINTED> = 1), <MINTED> (quartiles du tarif avec <MINTED>). Calculez le taux de survie pour chacune de ces nouvelles variables.

Exercice 5.5 (***). Construisez un pipeline complet de nettoyage pour le jeu <MINTED> : (a) suppression de <MINTED>, (b) imputation de <MINTED> et <MINTED>, (c) création de

<MINTED> et <MINTED>, (d) one-hot encoding de <MINTED>, <MINTED>, <MINTED>, (e) standardisation des variables numériques. Le résultat final doit être un DataFrame entièrement numérique sans valeurs manquantes.

Exercice 5.6 (★ ★ ★). Chargez le jeu <MINTED> (<MINTED>). Ce jeu contient des valeurs manquantes. Appliquez trois stratégies différentes (suppression, imputation par médiane, imputation par médiane conditionnée par espèce). Comparez les statistiques descriptives obtenues avec chaque stratégie et discutez de la meilleure approche.

Fonctions clés

Résumé du chapitre – Nettoyage et préparation

- Manquants : <MINTED>, types MCAR/MAR/MNAR, <MINTED>, <MINTED>.
- Doublons : <MINTED>, <MINTED>.
- Types : <MINTED>, <MINTED>, <MINTED>.
- Outliers : méthode IQR, écrêtage avec <MINTED>.
- Texte : <MINTED>, <MINTED>, <MINTED>.
- Feature engineering : <MINTED>, <MINTED>, combinaison de colonnes.
- Encodage : <MINTED> (one-hot), <MINTED> (ordinal).
- Mise à l'échelle : $z = \frac{x-\mu}{\sigma}$ (StandardScaler), $x' = \frac{x-x_{\min}}{x_{\max}-x_{\min}}$ (MinMaxScaler).
- Règle d'or : documenter chaque transformation et éviter le *data leakage*.

Chapitre 6

Statistiques Appliquées

Intuition

Les statistiques fournissent le cadre mathématique qui permet de passer de l'observation à l'inférence. En science des données, elles servent à quantifier l'incertitude, tester des hypothèses et mesurer les relations entre variables.

6.1 Statistiques descriptives

6.1.1 Mesures de tendance centrale et de dispersion

Définition 6.1 (Mesures fondamentales). Pour un échantillon $(x_1, x_2, \dots, x_n)$:

- **Moyenne** : $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
- **Médiane** : valeur telle que 50 % des observations sont inférieures.
- **Mode** : valeur la plus fréquente.
- **Variance** : $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$
- **Écart-type** : $s = \sqrt{s^2}$
- **Quantiles** : Q_1 (25 %), Q_2 (50 %), Q_3 (75 %).

Python

<MINTED>

Sortie

<MINTED>

Remarque 6.2. L'asymétrie positive (1.13) et la moyenne supérieure à la médiane confirment une distribution étirée vers la droite. Le kurtosis positif indique des queues plus lourdes que la loi normale.

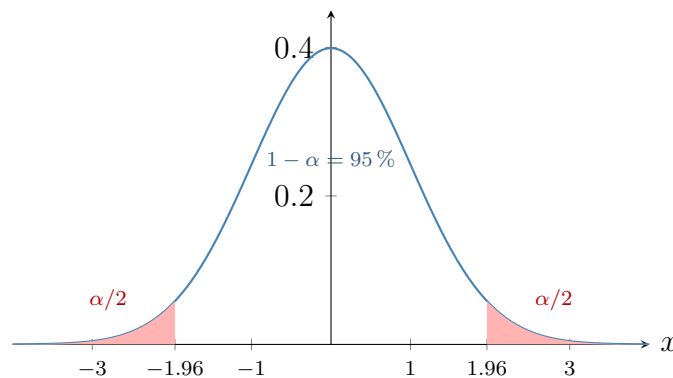
6.2 Distributions de probabilité

6.2.1 Loi normale

Définition 6.3 (Loi normale). Une variable aléatoire X suit une loi normale $\mathcal{N}(\mu, \sigma^2)$ si sa densité est :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Distribution normale $\mathcal{N}(0, 1)$ avec régions critiques



Python

<MINTED>

Sortie

<MINTED>

6.2.2 Loïs binomiale et de Poisson

Python

<MINTED>

Sortie

<MINTED>

6.3 Intervalles de confiance

Définition 6.4 (Intervalle de confiance). Un intervalle de confiance à $(1 - \alpha) \times 100\%$ pour la moyenne μ est :

$$\text{IC} = \left[\bar{x} - z_{\alpha/2} \frac{s}{\sqrt{n}}, \bar{x} + z_{\alpha/2} \frac{s}{\sqrt{n}} \right]$$

où $z_{\alpha/2}$ est le quantile de la loi normale (1.96 pour 95 %).

Python

<MINTED>

Sortie

<MINTED>

6.4 Tests d'hypothèses

Définition 6.5 (Test d'hypothèse). Un test d'hypothèse oppose :

- H_0 (hypothèse nulle) : pas d'effet, pas de différence.
- H_1 (hypothèse alternative) : il existe un effet ou une différence.

La **p-valeur** est la probabilité d'observer un résultat au moins aussi extrême que celui observé, sous H_0 . On rejette H_0 si $p < \alpha$ (généralement $\alpha = 0.05$).

6.4.1 Test t de Student

Python

<MINTED>

Sortie

<MINTED>

6.4.2 Test du χ^2 d'indépendance

Python

<MINTED>

Sortie

<MINTED>

Remarque 6.6. Le test du χ^2 confirme avec une énorme significativité statistique ($p \approx 10^{-58}$) que la survie dépendait fortement du sexe. C'est le résultat le plus marquant du Titanic : « Women and children first. »

6.5 Corrélations

Définition 6.7 (Coefficients de corrélation).

$$\text{néaire} : r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

- **Pearson** : mesure la corrélation *linéaire*.
- **Spearman** : mesure la corrélation *monotone* (basée sur les rangs).

Les deux coefficients sont dans $[-1, 1]$. $|r| > 0.7$ indique une corrélation forte.

Python

<MINTED>

Sortie

<MINTED>

Attention

Corrélation n'implique pas causalité. Une forte corrélation entre deux variables peut être due à une variable confondante. Toujours compléter par une analyse théorique et, si possible, un plan expérimental.

6.6 ANOVA à un facteur

Définition 6.8 (ANOVA). L'analyse de la variance (ANOVA) teste si les moyennes de k groupes sont égales :

- $H_0 : \mu_1 = \mu_2 = \dots = \mu_k$
- H_1 : au moins deux moyennes diffèrent.

La statistique F compare la variance inter-groupes à la variance intra-groupes.

Python

<MINTED>

Sortie

<MINTED>

Remarque 6.9. L'ANOVA ne détecte pas de différence significative ($p = 0.17$). Bien que le dimanche ait la moyenne la plus élevée, la variabilité intra-groupe est trop importante pour conclure.

6.7 Exemple intégrateur : analyse du Titanic

Python

<MINTED>

Sortie

<MINTED>

Proposition 6.10. Les résultats montrent que (1) le tarif diffère très significativement entre classes, (2) la survie dépend fortement de la classe, et (3) l'âge et le tarif ont une corrélation très faible bien que significative (l'effectif élevé rend de petits effets significatifs).

6.8 Exercices

Exercice 6.1 (*). Calculez la moyenne, la médiane, l'écart-type et les quartiles de la variable `<MINTED>` du jeu `<MINTED>`. La distribution est-elle symétrique ? Justifiez avec le coefficient d'asymétrie.

Exercice 6.2 (*). Générez 1000 échantillons de taille 30 à partir d'une loi $\mathcal{N}(50, 10^2)$. Pour chaque échantillon, calculez l'IC à 95 %. Combien d'intervalles contiennent effectivement $\mu = 50$? Le résultat est-il conforme à la théorie ?

Exercice 6.3 (**). Testez avec un test t si le pourboire moyen diffère entre les repas du midi (`<MINTED>`) et du soir (`<MINTED>`). Interprétez la p -valeur et calculez la taille d'effet (différence des moyennes divisée par l'écart-type poolé).

Exercice 6.4 (**). Réalisez un test du χ^2 pour tester l'indépendance entre `<MINTED>` et `<MINTED>` dans le jeu `<MINTED>`. Affichez la table de contingence, les effectifs théoriques et concluez.

Exercice 6.5 (***). Sur le jeu `<MINTED>`, réalisez les tests suivants et synthétisez les résultats dans un tableau : (a) t -test sur l'âge entre survivants et non-survivants, (b) χ^2 sur la survie vs le port d'embarquement, (c) ANOVA sur le tarif par port d'embarquement, (d) corrélation de Spearman entre `<MINTED>` et `<MINTED>`. Appliquez la correction de Bonferroni ($\alpha_{\text{corrigé}} = 0.05/4 = 0.0125$) et concluez.

Exercice 6.6 (***). Simulez la puissance d'un test t : générez deux groupes de taille n à partir de $\mathcal{N}(0, 1)$ et $\mathcal{N}(\delta, 1)$ avec $\delta \in \{0.2, 0.5, 0.8\}$ et $n \in \{10, 30, 100, 300\}$. Pour chaque combinaison, répétez 1000 fois et calculez le pourcentage de rejets de H_0 . Tracez un graphique de la puissance en fonction de n pour chaque δ .

Fonctions clés

Résumé du chapitre – Statistiques appliquées

- **Descriptif** : `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`.
- **Lois** : `<MINTED>`, `<MINTED>`, `<MINTED>` de `<MINTED>` ; méthodes `<MINTED>`, `<MINTED>`, `<MINTED>`.
- **IC à 95 %** : $\bar{x} \pm 1.96 \cdot \frac{s}{\sqrt{n}}$; exact avec `<MINTED>`.
- **Test t** : `<MINTED>` – compare deux moyennes indépendantes.
- **Test χ^2** : `<MINTED>` – indépendance de deux variables catégorielles.
- **Corrélation** : `<MINTED>`, `<MINTED>`, $r \in [-1, 1]$.
- **ANOVA** : `<MINTED>` – compare $k \geq 3$ moyennes.
- **Décision** : rejeter H_0 si $p < \alpha$ (souvent $\alpha = 0.05$).
- **Attention** : corrélation $\neq$ causalité ; significativité statistique $\neq$ importance pratique.

Chapitre 7

Introduction à l'Apprentissage Automatique

7.1 Qu'est-ce que l'apprentissage automatique ?

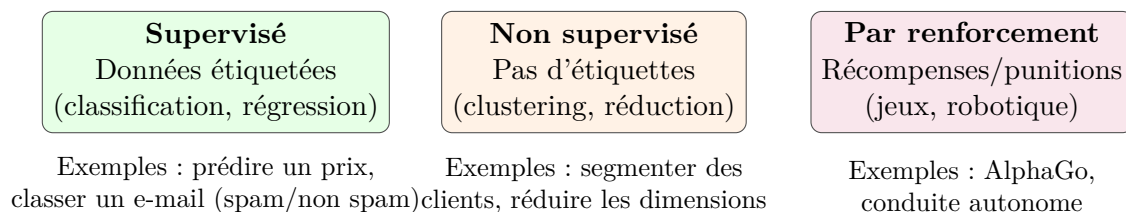
Définition 7.1 (Apprentissage automatique). L'**apprentissage automatique** (*machine learning*, ML) est une branche de l'intelligence artificielle qui permet aux ordinateurs d'apprendre à partir de données sans être explicitement programmés pour chaque tâche.

Intuition

Plutôt que d'écrire des règles manuellement (par exemple, « si le client a plus de 30 ans et un revenu supérieur à 50 000 €, alors approuver le prêt »), on fournit des **exemples** au modèle qui découvre lui-même les règles sous-jacentes.

7.1.1 Types d'apprentissage

Types d'apprentissage automatique



Définition 7.2 (Apprentissage supervisé). On dispose de paires $(\mathbf{x}_i, y_i)$ où $\mathbf{x}_i$ sont les **variables explicatives** (*features*) et y_i est la **cible** (*target*). Le modèle apprend une fonction f telle que $f(\mathbf{x}_i) \approx y_i$.

- **Classification** : y est une catégorie (spam/non spam, espèce de fleur).
- **Régression** : y est une valeur continue (prix, température).

Définition 7.3 (Apprentissage non supervisé). On dispose uniquement de $\mathbf{x}_i$ sans étiquettes. Le modèle cherche des **structures cachées** dans les données (groupes, axes principaux).

7.2 Le flux de travail en apprentissage supervisé



1. **Collecte et préparation** des données.
2. **Découpage** en ensemble d'entraînement et ensemble de test.
3. **Entraînement** du modèle sur les données d'entraînement.
4. **Prédiction** sur les données de test.
5. **Évaluation** des performances.

Python

<MINTED>

Sortie

<MINTED>

7.3 L'API Scikit-learn

Bonne pratique

Tous les modèles scikit-learn suivent la même interface :

1. <MINTED> — créer le modèle.
2. <MINTED> — entraîner.
3. <MINTED> — prédire.
4. <MINTED> — évaluer.

7.3.1 Exemple complet : k plus proches voisins sur Iris

Python

<MINTED>

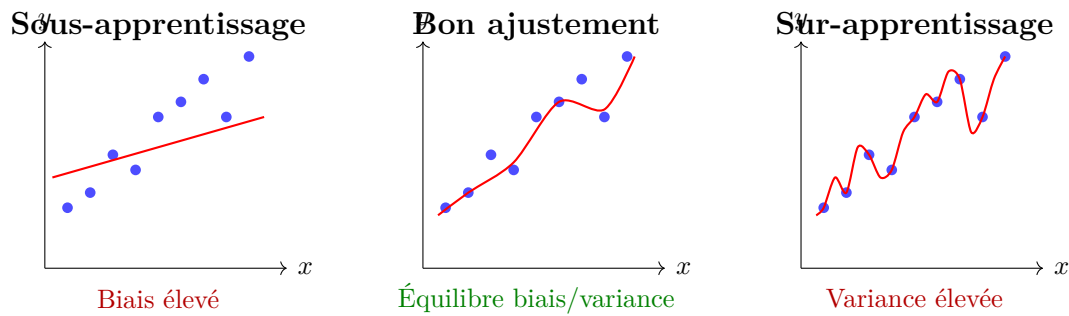
Sortie

<MINTED>

Remarque 7.4. La méthode <MINTED> renvoie l'**exactitude** (*accuracy*) pour la classification et le **coefficient de détermination** R^2 pour la régression.

7.4 Sur-apprentissage et sous-apprentissage

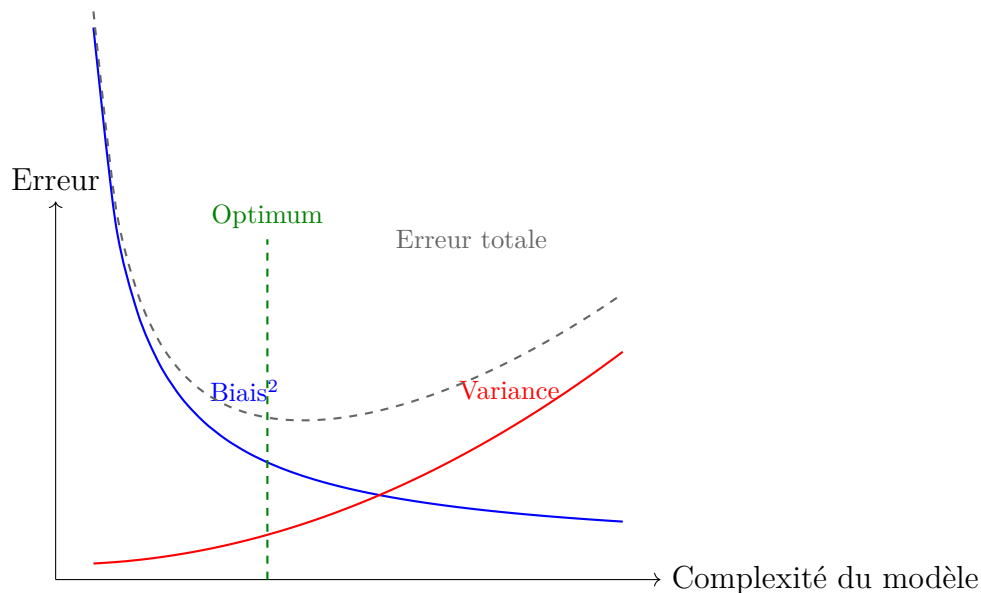
- Définition 7.5** (Sur-apprentissage et sous-apprentissage). • **Sous-apprentissage** (*underfitting*) : le modèle est trop simple, il ne capture pas la structure des données. **Biais élevé.**
- **Sur-apprentissage** (*overfitting*) : le modèle est trop complexe, il mémorise le bruit des données d'entraînement. **Variance élevée.**



Théorème 7.6 (Compromis biais-variance). *L'erreur de généralisation d'un modèle se décompose en :*

$$\text{Erreur totale} = \text{Biais}^2 + \text{Variance} + \text{Bruit irréductible}$$

Le but est de trouver le modèle qui minimise cette erreur totale, en équilibrant biais et variance.



7.5 Validation croisée

Définition 7.7 (Validation croisée à k plis). La **validation croisée** (*k-fold cross-validation*) divise les données en k sous-ensembles (*folds*). On entraîne le modèle k fois, chaque fois en utilisant un pli différent comme ensemble de test et les $k - 1$ autres comme ensemble d'entraînement.

Validation croisée à 5 plis

Pli 1	Test	Train	Train	Train	Train
Pli 2	Train	Test	Train	Train	Train
Pli 3	Train	Train	Test	Train	Train
Pli 4	Train	Train	Train	Test	Train
Pli 5	Train	Train	Train	Train	Test

Python

<MINTED>

Sortie

<MINTED>

Attention

Ne jamais évaluer un modèle sur les données d'entraînement ! Un score parfait sur l'entraînement peut cacher un **sur-apprentissage** sévère. Utilisez toujours un ensemble de test séparé ou la validation croisée.

7.5.1 Choix de k dans KNN

Python

<MINTED>

Sortie

<MINTED>

7.6 Exercices

Exercice 7.1 (*). Chargez le jeu de données <MINTED> de scikit-learn. Découpez-le en 80%/20%, entraînez un KNN avec $k = 3$ et affichez l'exactitude sur l'ensemble de test.

Exercice 7.2 (*). Expliquez avec vos propres mots la différence entre sur-apprentissage et sous-apprentissage. Donnez un exemple concret pour chacun.

Exercice 7.3 (**). Sur le jeu de données Iris, comparez les performances de KNN pour $k \in \{1, 3, 5, 7, 9, 11\}$ en utilisant la validation croisée à 10 plis. Tracez un graphique des scores moyens en fonction de k .

Exercice 7.4 (**). Chargez le jeu <MINTED> et réalisez une classification avec KNN. Calculez la validation croisée à 5 plis. Que remarquez-vous sur les performances ?

Exercice 7.5 (***). Chargez le jeu de données `penguins` de `seaborn`. Préparez les données (supprimez les valeurs manquantes, encodez les variables catégorielles avec `<MINTED>`). Entraînez un KNN pour prédire l'espèce et évaluez avec la validation croisée. Comparez les performances avec et sans normalisation des données (`<MINTED>`).

Exercice 7.6 (* * *). Créez un jeu de données synthétique avec `<MINTED>`. Comparez l'exactitude d'entraînement et de test pour un KNN avec $k = 1$ et $k = 10$. Lequel sur-apprend le plus ? Justifiez.

Fonctions clés

Résumé du chapitre ??

- **Apprentissage supervisé** : données étiquetées $(\mathbf{x}_i, y_i)$, classification ou régression.
- **Apprentissage non supervisé** : pas d'étiquettes, découverte de structures.
- **API scikit-learn** : `<MINTED>`, `<MINTED>`, `<MINTED>`.
- **Découpage** : `<MINTED>`.
- **Sur-apprentissage** : modèle trop complexe, variance élevée.
- **Sous-apprentissage** : modèle trop simple, biais élevé.
- **Compromis biais-variance** : $\text{Erreur} = \text{Biais}^2 + \text{Variance} + \text{Bruit}$.
- **Validation croisée** : `<MINTED>`, estimation robuste.

Chapitre 8

Modèles de Machine Learning

8.1 Régression linéaire

Définition 8.1 (Régression linéaire). La **régression linéaire** modélise la relation entre une variable cible y et des variables explicatives $\mathbf{x}$:

$$y = \mathbf{X}\boldsymbol{\beta} + \varepsilon$$

où $\boldsymbol{\beta}$ est le vecteur des coefficients et ε est le terme d'erreur.

Théorème 8.2 (Moindres carrés ordinaires (MCO)). *L'estimateur MCO minimise la somme des carrés des résidus :*

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Définition 8.3 (Coefficient de détermination R^2). Le R^2 mesure la proportion de la variance expliquée par le modèle :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \in (-\infty, 1]$$

Un R^2 proche de 1 indique un bon ajustement.

Python

<MINTED>

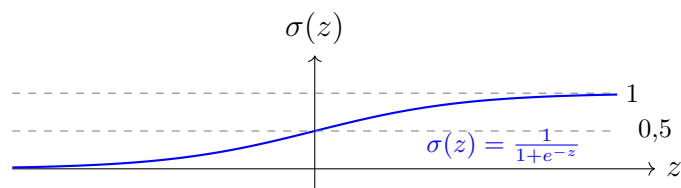
Sortie

<MINTED>

8.2 Régression logistique

Définition 8.4 (Régression logistique). La **régression logistique** est un modèle de **classification binaire**. Elle modélise la probabilité d'appartenance à la classe 1 via la fonction sigmoïde :

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}}$$



Python

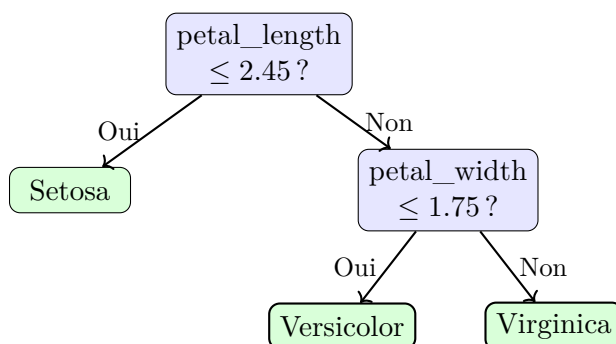
<MINTED>

Sortie

<MINTED>

8.3 Arbres de décision

Définition 8.5 (Arbre de décision). Un **arbre de décision** partitionne l'espace des variables explicatives par des tests binaires successifs, formant une structure arborescente. Chaque feuille correspond à une prédiction.



Python

<MINTED>

Sortie

<MINTED>

Attention

Un arbre sans limite de profondeur (<MINTED>) risque de **sur-apprendre**. Limitez la profondeur ou le nombre minimum d'échantillons par feuille (<MINTED>).

8.4 Forêts aléatoires

Définition 8.6 (Forêt aléatoire). Une **forêt aléatoire** (*Random Forest*) est un **ensemble** de B arbres de décision entraînés sur des sous-échantillons aléatoires des données (*bagging*). La prédiction finale est le **vote majoritaire** (classification) ou la **moyenne** (régression).

Python

<MINTED>

Sortie

<MINTED>

8.5 K plus proches voisins (KNN)

Définition 8.7 (KNN). Le classifieur ***k* plus proches voisins** attribue à une nouvelle observation la classe majoritaire parmi ses *k* voisins les plus proches selon une métrique de distance (généralement euclidienne) :

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

Bonne pratique

La performance de KNN dépend fortement de l'**échelle** des variables. Normalisez toujours vos données avant d'utiliser KNN :

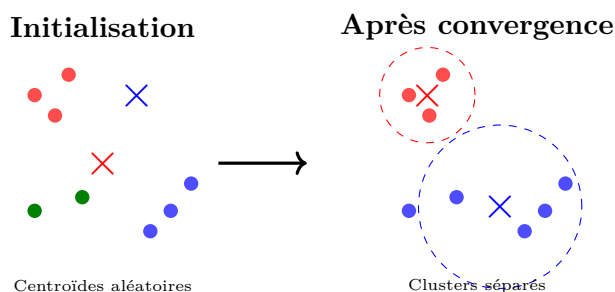
<MINTED>

8.6 K-Means : clustering

Définition 8.8 (K-Means). **K-Means** est un algorithme d'apprentissage **non supervisé** qui partitionne *n* observations en *K* groupes (*clusters*) en minimisant l'inertie :

$$\mathcal{J} = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

où $\boldsymbol{\mu}_k$ est le centroïde du cluster C_k .



Python

<MINTED>

Sortie

<MINTED>

Intuition

La **méthode du coude** consiste à chercher le point où l'inertie cesse de diminuer significativement. Ici, $K = 3$ correspond au « coude », cohérent avec les 3 espèces d'Iris.

8.7 Tableau comparatif des modèles

Modèle	Type	Interprétable	Normalisation	Hyperparamètres clés
Rég. linéaire	Régression	Oui	Non	—
Rég. logistique	Classification	Oui	Oui	<MINTED>
Arbre de décision	Les deux	Oui	Non	<MINTED>
Forêt aléatoire	Les deux	Non	Non	<MINTED>, <MINTED>
KNN	Les deux	Non	Oui	<MINTED>
K-Means	Clustering	Moyen	Oui	<MINTED>

8.8 Exercices

Exercice 8.1 (*). Chargez le jeu de données `tips` de seaborn. Entraînez une régression linéaire pour prédire le pourboire (<MINTED>) à partir de <MINTED> uniquement. Affichez le coefficient, l'intercept et le R^2 .

Exercice 8.2 (*). Sur le jeu Iris, comparez l'exactitude (validation croisée à 5 plis) d'un arbre de décision (<MINTED>), d'une forêt aléatoire (100 arbres) et de KNN ($k = 5$).

Exercice 8.3 (**). Chargez le jeu `titanic` de seaborn. Préparez les données (supprimez les lignes avec valeurs manquantes dans <MINTED>, <MINTED>; utilisez <MINTED>, <MINTED>, <MINTED>, <MINTED> encodé). Comparez la régression logistique et une forêt aléatoire pour prédire <MINTED>.

Exercice 8.4 (**). Appliquez K-Means avec $K = 3$ sur les données Iris (sans les étiquettes). Comparez les clusters obtenus avec les vraies espèces à l'aide d'un tableau croisé (<MINTED>).

Exercice 8.5 (* * *). Chargez le jeu <MINTED> de scikit-learn. Entraînez une régression linéaire et une forêt aléatoire. Comparez les R^2 et RMSE sur l'ensemble de test. Tracez les prédictions vs. les valeurs réelles pour les deux modèles.

Exercice 8.6 (* * *). Implémentez l'algorithme KNN « à la main » (sans scikit-learn) en utilisant NumPy. Testez-le sur Iris et comparez vos résultats avec <MINTED>.

Fonctions clés

Résumé du chapitre ??

- **Régression linéaire** : $y = \mathbf{X}\boldsymbol{\beta} + \varepsilon$, MCO : $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.
- **Régression logistique** : $P(y=1|\mathbf{x}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta})$, classification binaire.
- **Arbre de décision** : partitions successives, contrôler **<MINTED>**.
- **Forêt aléatoire** : ensemble d'arbres par *bagging*, plus robuste.
- **KNN** : vote des k plus proches voisins, nécessite normalisation.
- **K-Means** : clustering non supervisé, minimise l'inertie $\mathcal{J}$.
- **Méthode du coude** : choisir K au point d'inflexion de l'inertie.

Chapitre 9

Évaluation des Modèles

9.1 Métriques de classification

Définition 9.1 (Matrice de confusion). La **matrice de confusion** résume les prédictions d'un classifieur binaire en quatre catégories :

- **Vrais positifs** (VP / TP) : correctement prédits positifs.
- **Faux positifs** (FP) : incorrectement prédits positifs (erreur de type I).
- **Vrais négatifs** (VN / TN) : correctement prédits négatifs.
- **Faux négatifs** (FN) : incorrectement prédits négatifs (erreur de type II).

		Matrice de confusion	
		Classe prédite	
Classe réelle		Positif	Négatif
	Positif	VP (TP)	FN
	Négatif	FP	VN (TN)

Définition 9.2 (Métriques de classification). À partir de la matrice de confusion, on définit :

$$\text{Exactitude (Accuracy)} = \frac{VP + VN}{VP + VN + FP + FN} \quad (9.1)$$

$$\text{Précision} = \frac{VP}{VP + FP} \quad (\text{parmi les prédits positifs}) \quad (9.2)$$

$$\text{Rappel (Recall)} = \frac{VP}{VP + FN} \quad (\text{parmi les vrais positifs}) \quad (9.3)$$

$$\text{F1-score} = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (9.4)$$

Intuition

- La **précision** répond à : « Parmi ceux que j'ai déclarés malades, combien le sont vraiment ? »
- Le **rappel** répond à : « Parmi les vrais malades, combien ai-je détectés ? »
- Le **F1-score** est la moyenne harmonique des deux, utile quand les classes sont déséquilibrées.

9.1.1 Exemple avec le Titanic**Python**

<MINTED>

Sortie

<MINTED>

Attention

L'**exactitude** seule est trompeuse sur des données déséquilibrées. Si 95% des e-mails sont légitimes, un modèle qui prédit toujours « légitime » aura 95% d'exactitude mais ne détectera aucun spam !

9.2 Courbe ROC et AUC

Définition 9.3 (Courbe ROC). La **courbe ROC** (*Receiver Operating Characteristic*) trace le **taux de vrais positifs** (rappel) en fonction du **taux de faux positifs** pour différents seuils de décision. L'**AUC** (*Area Under the Curve*) mesure l'aire sous cette courbe : $AUC \in [0, 1]$.

Python

<MINTED>

Sortie

<MINTED>

Remarque 9.4. Un classifieur aléatoire a une AUC de 0,5 (diagonale). Un classifieur parfait a une AUC de 1,0. En général, une $AUC > 0,8$ est considérée comme bonne.

9.3 Métriques de régression

Définition 9.5 (Métriques de régression). Soit $\hat{y}_i$ les prédictions et y_i les valeurs réelles :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (9.5)$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (9.6)$$

$$\text{RMSE} = \sqrt{\text{MSE}} \quad (9.7)$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (9.8)$$

Python

<MINTED>

Sortie

<MINTED>

9.4 Validation croisée avancée

Définition 9.6 (Validation croisée stratifiée). La **validation croisée stratifiée** préserve la **proportion des classes** dans chaque pli. Elle est recommandée pour la classification, surtout avec des classes déséquilibrées.

Python

<MINTED>

Sortie

<MINTED>

9.5 Optimisation des hyperparamètres

Définition 9.7 (GridSearchCV). <MINTED> effectue une **recherche exhaustive** sur une grille d'hyperparamètres, en évaluant chaque combinaison par validation croisée.

Python

<MINTED>

Sortie

<MINTED>

Bonne pratique

Utilisez **<MINTED>** pour trouver les meilleurs hyperparamètres de manière systématique. Pour de grandes grilles, préférez **<MINTED>** qui échantillonne aléatoirement un sous-ensemble de combinaisons.

9.6 Courbes d'apprentissage

Définition 9.8 (Courbe d'apprentissage). Une **courbe d'apprentissage** trace le score d'entraînement et de validation en fonction de la **taille de l'ensemble d'entraînement**. Elle permet de diagnostiquer le sur-apprentissage ou le sous-apprentissage.

Python

<MINTED>

Sortie

<MINTED>

Intuition

- Si les scores d'entraînement et de validation sont **proches et bas** : sous-apprentissage (augmenter la complexité).
- Si le score d'entraînement est **élevé** mais le score de validation est **bas** : sur-apprentissage (réduire la complexité ou augmenter les données).
- Si les deux scores **convergent vers une valeur élevée** : bon modèle.

9.7 Exercices

Exercice 9.1 (★). Calculez manuellement la précision, le rappel et le F1-score à partir de la matrice de confusion suivante :

$$\begin{pmatrix} 45 & 5 \\ 10 & 40 \end{pmatrix}$$

Vérifiez vos résultats avec scikit-learn.

Exercice 9.2 (★). Expliquez dans quel cas on privilégie le rappel plutôt que la précision. Donnez deux exemples concrets.

Exercice 9.3 (★★). Sur le jeu Iris, entraînez un KNN et tracez la courbe ROC (one-vs-rest) pour chaque classe. Calculez l'AUC pour chaque classe.

Exercice 9.4 (★★). Utilisez **<MINTED>** pour optimiser les hyperparamètres d'un **<MINTED>** sur le jeu de données **penguins** de seaborn. Testez $k \in \{1, 3, 5, 7, 9, 11\}$ et les métriques de distance **<MINTED>**.

Exercice 9.5 (***). Sur le jeu Titanic, comparez trois modèles (régression logistique, forêt aléatoire, KNN) en utilisant la validation croisée stratifiée à 10 plis. Pour chaque modèle, rapportez l’exactitude, la précision, le rappel et le F1-score moyens. Quel modèle recommandez-vous et pourquoi ?

Exercice 9.6 (***). Tracez les courbes d’apprentissage pour un arbre de décision avec **<MINTED>**, **<MINTED>** et **<MINTED>** sur le jeu Titanic. Interprétez les résultats en termes de biais-variance.

Fonctions clés

Résumé du chapitre ??

- **Matrice de confusion** : VP, FP, VN, FN.
- **Exactitude** : $(VP + VN)/N$, trompeuse si classes déséquilibrées.
- **Précision** : $VP/(VP + FP)$, **Rappel** : $VP/(VP + FN)$.
- **F1-score** : $2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$.
- **Courbe ROC / AUC** : TPR vs. FPR, $AUC \in [0, 1]$.
- **Régression** : MAE, MSE, RMSE, R^2 .
- **GridSearchCV** : recherche exhaustive d’hyperparamètres avec CV.
- **Courbe d’apprentissage** : diagnostic sur-/sous-apprentissage.

Chapitre 10

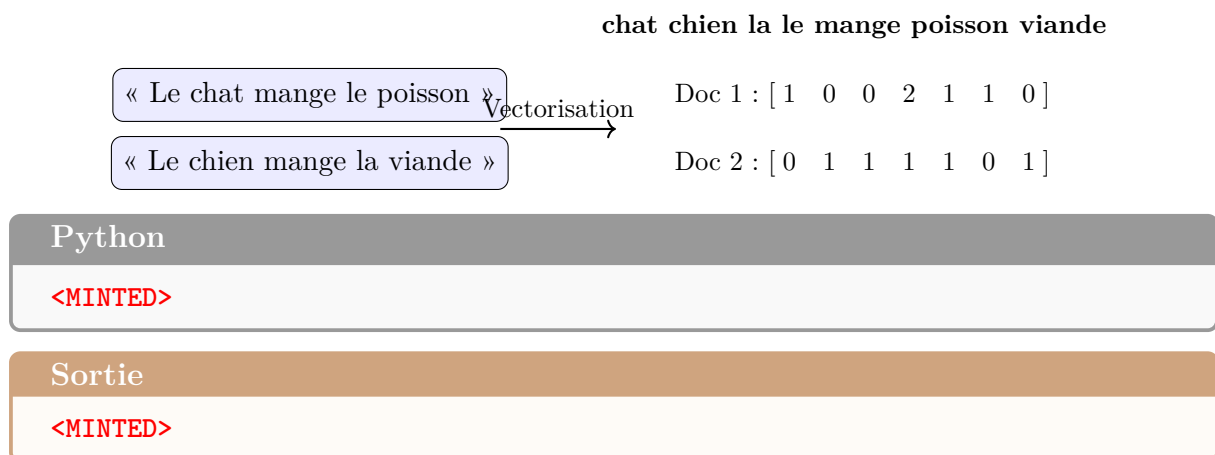
Introduction au Texte et aux Séries Temporelles

10.1 Données textuelles

Définition 10.1 (Traitement automatique du langage). Le **traitement automatique du langage naturel** (*Natural Language Processing*, NLP) regroupe les techniques permettant aux machines de comprendre et manipuler du texte. Pour utiliser du texte en apprentissage automatique, il faut le convertir en **représentation numérique**.

10.1.1 Sac de mots (*Bag of Words*)

Définition 10.2 (Sac de mots). Le modèle **sac de mots** représente chaque document comme un vecteur de fréquences de mots, en ignorant l'ordre et la grammaire.



10.1.2 TF-IDF

Définition 10.3 (TF-IDF). **TF-IDF** (*Term Frequency–Inverse Document Frequency*) pondère chaque terme par son importance :

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

où $\text{TF}(t, d)$ est la fréquence du terme t dans le document d et :

$$\text{IDF}(t) = \log \frac{n_{\text{docs}}}{n_{\text{docs contenant } t}}$$

Les mots fréquents partout (comme « le ») reçoivent un poids faible.

Python

<MINTED>

Sortie

<MINTED>

Intuition

Les mots « poisson » et « viande » ont des poids TF-IDF élevés car ils sont **discriminants** : ils n'apparaissent que dans un seul document. Le mot « le », présent partout, a un poids relativement plus faible.

10.1.3 Classification de texte

Python

<MINTED>

Sortie

<MINTED>

Bonne pratique

Utilisez <MINTED> pour chaîner le prétraitement et le modèle. Cela garantit que la vectorisation est apprise uniquement sur les données d'entraînement, évitant les fuites de données (*data leakage*).

10.2 Séries temporelles

Définition 10.4 (Série temporelle). Une **série temporelle** est une suite d'observations $(y_{t_1}, y_{t_2}, \dots, y_{t_n})$ indexées par le temps. L'analyse des séries temporelles vise à comprendre les tendances, la saisonnalité et à effectuer des prévisions.

10.2.1 Manipulation des dates avec pandas

Python

<MINTED>

Sortie

<MINTED>

10.2.2 L'accessor `.dt`

Python

<MINTED>

Sortie

<MINTED>

10.2.3 Ré-échantillonnage et moyennes mobiles

Python

<MINTED>

Sortie

<MINTED>

10.2.4 Décomposition d'une série temporelle

Définition 10.5 (Décomposition). Une série temporelle peut être décomposée en trois composantes :

$$y_t = T_t + S_t + R_t$$

où T_t est la **tendance**, S_t la **saisonnalité** et R_t le **résidu**.

Python

<MINTED>

Sortie

<MINTED>

Remarque 10.6. La décomposition peut être **additive** ($y_t = T_t + S_t + R_t$) ou **multiplicative** ($y_t = T_t \times S_t \times R_t$). Le modèle multiplicatif est adapté lorsque l'amplitude de la saisonnalité croît avec la tendance.

10.3 Exercices

Exercice 10.1 (★). Créez un <MINTED> sur les phrases suivantes : « J'aime le Python », « J'aime les statistiques », « Python et statistiques sont utiles ». Affichez la matrice document-terme et le vocabulaire.

Exercice 10.2 (★★). Chargez le jeu de données <MINTED> avec les catégories <MINTED>. Construisez un pipeline TF-IDF + régression logistique. Évaluez avec la validation croisée à 5 plis et affichez le rapport de classification.

Exercice 10.3 (★★). Créez une série temporelle quotidienne sur 3 ans avec une tendance linéaire et une saisonnalité hebdomadaire. Ré-échantillonnez par semaine et par mois. Tracez les trois graphiques (quotidien, hebdomadaire, mensuel).

Exercice 10.4 (***). Chargez un jeu de données temporelles réel (par exemple, les données de température de `<MINTED>`). Appliquez la décomposition saisonnière. Tracez les quatre composantes (série originale, tendance, saisonnalité, résidu) sur des sous-graphiques.

Exercice 10.5 (***). Comparez les performances de classification de texte en utilisant (a) `<MINTED>`, (b) `<MINTED>` et (c) `<MINTED>` sur le jeu 20 newsgroups (4 catégories au choix). Utilisez un classifieur `<MINTED>` et rapportez les exactitudes.

Fonctions clés

Résumé du chapitre ??

- **Sac de mots** : `<MINTED>`, matrice de fréquences de termes.
- **TF-IDF** : `<MINTED>`, $\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{n_{\text{docs}}}{n_{\text{docs}(t)}}$.
- **Pipeline** : `<MINTED>`.
- **Dates pandas** : `<MINTED>`, accesseur `<MINTED>` (year, month, day_name).
- **Ré-échantillonnage** : `<MINTED>`.
- **Moyenne mobile** : `<MINTED>`.
- **Décomposition** : $y_t = T_t + S_t + R_t$ via `<MINTED>`.

11.2 Projet 1 : Prédiction de survie sur le Titanic

11.2.1 Compréhension du problème

L'objectif est de prédire la **survie** des passagers du Titanic à partir de leurs caractéristiques (classe, âge, sexe, tarif). C'est un problème de **classification binaire**.

11.2.2 Étape 1 : Chargement et exploration

Python

<MINTED>

Sortie

<MINTED>

Python

<MINTED>

Sortie

<MINTED>

11.2.3 Étape 2 : Préparation des données

Python

<MINTED>

Sortie

<MINTED>

11.2.4 Étape 3 : Modélisation et évaluation

Python

<MINTED>

Sortie

<MINTED>

11.2.5 Étape 4 : Évaluation finale

Python

<MINTED>

Sortie

<MINTED>

Python

<MINTED>

Sortie

<MINTED>

Remarque 11.2. Le **sexe** est de loin la variable la plus importante pour prédire la survie, suivi du **tarif** et de l'**âge**. Cela reflète la règle « femmes et enfants d'abord » appliquée lors de l'évacuation.

11.3 Projet 2 : Prédiction du prix immobilier en Californie

11.3.1 Compréhension du problème

L'objectif est de prédire le **prix médian des logements** en Californie à partir de caractéristiques géographiques et démographiques. C'est un problème de **régression**.

11.3.2 Étape 1 : Chargement et exploration

Python

<MINTED>

Sortie

<MINTED>

11.3.3 Étape 2 : Préparation et découpage

Python

<MINTED>

Sortie

<MINTED>

11.3.4 Étape 3 : Modélisation

Python

<MINTED>

Sortie

<MINTED>

11.3.5 Étape 4 : Analyse des résultats

Python

<MINTED>

Sortie

<MINTED>

Python

<MINTED>

Sortie

<MINTED>

Intuition

Le **revenu médian** (<MINTED>) est de loin le meilleur prédicteur du prix immobilier, suivi de la **localisation géographique** (latitude/longitude). La forêt aléatoire ($R^2 \approx 0,81$) améliore nettement la régression linéaire ($R^2 \approx 0,58$) grâce à sa capacité à capturer les relations non linéaires.

11.4 Conseils pour présenter vos résultats

Bonne pratique

Pour un projet de science des données réussi :

1. **Structurez votre rapport** selon CRISP-DM ou un plan similaire.
2. **Documentez chaque étape** : choix des variables, traitement des valeurs manquantes, paramètres des modèles.
3. **Visualisez les résultats** : graphiques clairs avec titres et axes légendés.
4. **Comparez plusieurs modèles** : tableau récapitulatif des métriques.
5. **Interprétez** : ne vous contentez pas des chiffres, expliquez ce que les résultats signifient dans le contexte métier.
6. **Soyez honnêtes** : mentionnez les limites de votre analyse.

Attention

Les erreurs les plus fréquentes dans les projets étudiants :

- Évaluer sur les données d'entraînement (fuite de données).
- Ignorer les valeurs manquantes sans justification.
- Choisir un modèle sans comparer d'alternatives.
- Présenter des résultats sans interprétation.
- Ne pas fixer la graine aléatoire (<MINTED>), rendant les résultats non reproductibles.

11.5 Exercices de projets

Exercice 11.1 (★★ — Projet guidé). **Classification des espèces de manchots.** Chargez le jeu de données `penguins` de `seaborn`. Suivez les étapes CRISP-DM :

1. Explorez les données (statistiques descriptives, valeurs manquantes).
2. Préparez les données (imputation, encodage).
3. Comparez au moins 3 modèles (KNN, arbre, forêt).
4. Évaluez avec la validation croisée et un rapport de classification.
5. Identifiez les variables les plus importantes.

Exercice 11.2 (★★★ — Projet avancé). **Prédiction de la qualité du vin.** Utilisez le jeu de données <MINTED> de `scikit-learn`. Objectif : prédire la catégorie du vin.

1. Explorez les corrélations entre les variables.
2. Testez la normalisation (<MINTED>) et son impact.
3. Comparez régression logistique, forêt aléatoire et KNN.
4. Optimisez les hyperparamètres avec <MINTED>.
5. Présentez les résultats sous forme de tableau et de graphiques.

Exercice 11.3 (★★★ — Projet ouvert). **Analyse complète d'un jeu de données au choix.** Choisissez un jeu de données parmi ceux disponibles dans `seaborn` (`diamonds`, `mpg`, `flights`) ou `scikit-learn`. Réalisez un projet complet en suivant la méthodologie CRISP-DM :

- Définissez clairement la question de recherche.
- Effectuez une analyse exploratoire approfondie.
- Entraînez et comparez au moins 2 modèles.
- Rédigez un rapport structuré (5 pages minimum) avec visualisations.

Fonctions clés**Résumé du chapitre ??**

- **CRISP-DM** : compréhension métier → données → préparation → modélisation → évaluation → déploiement.
- **Pipeline type** : charger, explorer, nettoyer, découper, modéliser, évaluer, interpréter.
- **Titanic** (classification) : sexe et tarif sont les variables clés, forêt aléatoire $\approx 80\%$ d'exactitude.
- **California Housing** (régression) : revenu médian est le meilleur prédicteur, forêt aléatoire $R^2 \approx 0,81$.
- **Bonnes pratiques** : reproductibilité (**<MINTED>**), comparaison de modèles, interprétation métier.

Référence rapide Python

.1 Pandas

Fonction	Description
<MINTED>	Charger un CSV
<MINTED>	Premiers rangs
<MINTED>	Statistiques descriptives
<MINTED>	Agréger par groupes
<MINTED>	Joindre deux DataFrames
<MINTED>	Tableau croisé dynamique
<MINTED> / <MINTED>	Gérer les valeurs manquantes

.2 Visualisation

Fonction	Description
<MINTED>	Graphique en ligne
<MINTED>	Nuage de points
<MINTED>	Histogramme
<MINTED>	Diagramme en barres
<MINTED>	Carte de chaleur
<MINTED>	Matrice de nuages

.3 Scikit-learn

Classe / Fonction	Description
<MINTED>	Diviser train/test
<MINTED>	Régression linéaire
<MINTED>	Régression logistique
<MINTED>	k -plus proches voisins
<MINTED>	Arbre de décision
<MINTED>	Clustering k -means
<MINTED>	Validation croisée

Bibliographie

- [1] VANDERPLAS, J. (2016). *Python Data Science Handbook*. O'Reilly.
- [2] MCKINNEY, W. (2022). *Python for Data Analysis*. 3e éd., O'Reilly.
- [3] GÉRON, A. (2022). *Hands-On Machine Learning*. 3e éd., O'Reilly.
- [4] JAMES, G. et al. (2021). *An Introduction to Statistical Learning*. 2e éd., Springer.